

JAVASCRIPT

JS + Functions + Expressions

Function Declaration: Schlüsselwort `function`, gehölst *Verwendung für Definition möglich*, `function greet() {}`.

Function Expression: in JSX und überall wo eine Expression reinpasst nutzbar (`()`)
Named Function Expr: `const greet = function greet(Person) {} ...`;
Anonymous Function Expr: `... = function () {} ...`;

Arrow Function (Lambda): kompakte Form einer Function Expression, 1 Parameter: `... = x => x * x`; mit Rückgabewert: `= () => "Hello"`;

Callback Function: (relevant in Express Middleware) `TODD`

Was sind Expressions? Expressions sind Werte oder Kombinationen von Werten, Variablen und Operatoren, die ein Resultat ergeben. Alles was etwas zurück gibt.

Expressions: Wie man Werte im JSX ausgeben kann u.v.m.

keine Expressions: `const a = [1,2]`, statements wie `if` sind keine Expression

Spread Syntax: ... , kopiert Array und Objekt Inhalte. Nur 1 Level tief.

Arrays im State ändern: `add [...arr]`, `remove`, `filter`, `slice`, `replacing`, `map`, `sorting`, `reversed`, `sorted`

REACT

REACT-REGELN (KOMPONENTE, HTML, JSX)

- ☐ Nur ein Rückgabewert: 1 parent element (Root Element) oder `React.Fragment <>`
- ☐ Naming of Komponente
 - UpperCamelCase
 - treffende Beschreibung der Komponente
 - Beispiele: `LinkButton`, `InfoTooltip`, `DraftEditor`
- ☐ HTML Attribute in JSX mit camelCase (außer `aria-*` und `data-*`)
 - `class => className`, `for` in Formularfeldern `> htmlFor`, `stroke-width` `> strokeLineidth`
 - HTML Elemente geschlossen, z.B. `<img />`, `<input />`
 - Syntax wie in JS: Wert in (title)
 - Keys bei Wiederholungen, z.B. `map()`

React Code lesen und Lernziele Inhalt

wieso React? SPA, interaktiv, JSX (intuitiver als JS für HTML)

Imperatives UI: beschreiben wie es geändert wird, **Deklaratives UI:** beschreiben wie es Aussehen soll

Pure Components: bei gleichen Props und gleichem State immer gleiche Resultate. React geht davon aus, dass Komponenten Pure sind.

Inline Style: `<div style={{ backgroundColor: "red" }}>Text</div>`

JSX: ist die Syntax, die wir für React Komponenten verwenden. Dabei wird JS und HTML miteinander verflochten. Syntax für React Komponente. Wird übersetzt in JS, JavaScript XML

Rendering:

Rendering: wenn sich state, props oder context von Komponente ändern.

sicheres State Update: weil in Eventhandler und Funktion wartet React bis ausgeführt worden, bevor State Updates verarbeitet werden. `setCount(prev => prev + 1)`;

React rendering Fehler:

```
<p>{obj}</p> /* Error "Objects are not valid as a React child" */
<p>{fn}</p> /* Error "Functions are not valid as a React child" */
```

Persistence

Möglichkeiten für Persistence: `localStorage`, `URL` Parameter, `sessionStorage`, `Cookies`, `indexedDB`, externe `DB/API`. Sensible Daten (z.B. Kreditkarteninfos) nur auf externer `DB/API` geeignet.

localStorage:

Speichert Daten dauerhaft im Browser + ist isoliert nach Seite, aber für mehrere Sessions (Tabs/Fenster) übergreifend verfügbar. Daten bleiben erhalten nach Page Reload und Neustart (kein Ablaufdatum). Ist ein einfacher key/value Speicher mit Strings (Typ: `JSON.stringify()` für Objekte). 5-10 MB Platz. Anwendung: Warenkorb oder Einstellungen.

`localStorage.getItem("theme")`, `localStorage.removeItem("theme")`, `localStorage.setItem("theme", "dark")`, `localStorage.clear()`;

```
// localStorage React useState:
const [theme, setTheme] = useState(
  localStorage.getItem("theme") || "light"
);
// Bein ändern speichern:
toggleTheme() => {
  setTheme(!theme);
  localStorage.setItem("theme", !theme);
};
```

sessionStorage:

Daten existieren nur während der Browser Session `Tab` oder `Browser geschlossen` `> Daten gelöscht`, `Page Reload` `> Daten nach da`. Isoliert nach einer Seite und eines Tabs/Fenster. API gleich wie bei `localStorage` (`getItem`, `removeItem`, `setItem`, `clear`)

Cookies (siehe Security)

kleine Datenstücke, die es ermöglichen, Daten zu speichern und zwischen Server und Client mitzugeben.

nur kleine Datenmenge, können Ablaufdatum haben, können serverseitig gelesen werden.

Anwendung: `Login Sessions`, `Tracking/Analytics`

IndexedDB

In jedem Browser eingebaute `NoSQL DB`. Für grössere und strukturierte Datenmengen geeignet. Speichert Objekte, komplexere API, sehr grosse Datenmengen möglich (50-250MB), theoretisch unbegrenzte Lebenszeit. **Anwendung:** Offline Apps, PWA, grosse Datensätze im Browser

Externe Datenbank / API

Daten können auch ausserhalb des Browsers gespeichert werden, zbsp. in 'Cloud' Datenbank (DBaaS) oder Backend APIs. **Vorteile:** Daten für viele User speicherbar, Mehr Logik und Datenerarbeitung möglich, Synchronisation zwischen Geräten, Kommunikation zwischen Nutzern

URL

JS Array map und filter + list und conditional rendering

List Rendering: Eine Anzahl gleicher Komponenten darstellen, aber mit unterschiedlichen Inhalten .`map()`

Conditional Rendering: Das Layout anpassen .`filter()`

```
arr2.map(x => 2*x);
arr2.filter(x => x === 'a');
const arr = ['a', 'b', 'c'];
arr.push("d"); //['a','b','c','d']
arr.forEach((e, i) => console.log(i + "="+e));
arr.sort((a,b) => a=b).map(e=>`<li>${e}</li>`).join('');
// list rendering:
<list.map((item) => {
  <PacklistItem key={item} item={item} />
})
// conditional rendering:
(isInStock ? ' ja' : ' nâ') <isInStock && 'ja'>
```

KOMPONENTEN, JSX, PROPS

Props (Properties)

Props sind Eingabewerte für Komponenten (Tatsächlich ist das Props Objekt der einzige akzeptierte Parameter)

- als Attribute übergeben und in Komponentenfunktion als Parameter auslesen
- Props können immer nur von Parent zum Child übergeben werden
- immutable
- Es können nicht nur Strings, sondern alle JS Datentypen als Props übergeben werden, auch andere Komponenten, JSX-Fragmente, Objekte, Listen, oder Funktionen.

STATE/ZUSTAND (USESTATE), EVENTHANDLER

Eventhandler

Wir brauchen Funktionen, die Interaktionen im UI mit dem State verbinden $\rightarrow$ Eventhandler. `<button onClick={() => setCounter(counter+1)}>Click Me</button>` Native HTML Elemente haben in React spezielle Attribute/Props, die Eventhandler aufnehmen. Einige der häufig verwendeten: `onClick` für Buttons `onChange` für Inputfelder `onFocus`/`onBlur` Diesen Attributen können wir Eventhandler-Funktionen übergeben. Eventhandler-Funktionen bekommen ein Eventobjekt `e`, das analog zum `Event` in JS funktioniert und Informationen zum Event enthält

Es gibt verschiedene Möglichkeiten, Eventhandler zu platzieren:

```
// separat + für Längere Handler, oder mehrfach verwendete (Demo 1)
const handleClick = () => alert('submitted')
return (
  <button onClick={handleClick}>Click Me</button>
)
// inline + für kurze Handler
return ( <button onClick={() => alert("submitted")}>Click Me</button> )
```

`// falsch`
`function handleClick() {<button onClick={handleClick}>Click Me</button> }`

Eventhandlers als Props übergeben

Häufig, wie in der Demo, übergeben wir Eventhandler als Props an Komponenten, um State in höhergelegenen Komponenten zu aktualisieren.

```
<SearchInput
  value={searchText}
  onChange={(e) => setSearchText(e.target.value)}
/>
```

Üblicherweise werden diese Props analog der nativen Eventhandler-Props mit `onThingThatHappens` benannt, wie `onUpdateSearchText` oder `onCloseDialog`

```
useState
function FilterComp() {
  const [name, setName] = useState("Max");
  return (
    <div>
      <input type="text" value={name} />
      <button onClick={setName} />
    </div>
  );
}

// falsch
function KindComp(props) {
  const { name, onupdate } = props;
  return <div>
    <input type="text" value={name} />
    <button onClick={onupdate(e => update(e.target.value))} />
  </div>;
}
```

weitere Hooks

`useRef`: um DOM-Nodes zu tracken und damit zum Beispiel **Fokus-Management** zu betreiben. Änderung von Wert triggered nicht renderen.

useReducer

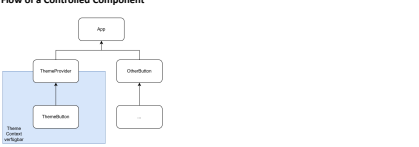
`useReducer` z.B. für **Tennismatch-Scoring**, komplexe Zustandslogik.

```
type ReducerActionTypes = 'increment' | 'decrement';
const reducer = (
  state: { count: number }, // state ist Zahl von counter
  action: { type: ReducerActionTypes }, // + oder -
) => {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      throw new Error('Unknown action type');
  }
};

export const Counter = () => {
  // use reducer with useReducer hook
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <div>
      <h3>useReducer</h3>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'decrement' })}</button>
    </div>;
  );
};
```

CONTROLLED FORMS

Flow of a Controlled Component



```
function Form() {
  const [formData, setFormData] = useState({
    firstName: '',
    lastName: '',
    email: '',
  });
  // generalized changehandler
  const handleInputChange = (e: React.ChangeEvent<HTMLFormElement>): void => {
    const { name, value } = e.target;
    setFormData(prevState => ({ ...prevState, [name]: value }));
  };
  // oder mit typed eventhandler statt typed event <
  const handleInputChange: React.ChangeEventHandler<HTMLFormElement> = (e) => {
    // ...
    <label htmlFor="firstName">First Name</label>
    <input type="text" name="firstName" id="firstName"
      value={formData.firstName} onChange={handleChange}
    />
  }
}
```

TYPESCRIPT

Typisierung von Objekten, Props, Funktionen.

typeof: für Type narrowing (`typeof val === "string"`) (und Typ Inferenz von TypeScript), mögliche Typen: `string`, `number`, `bigint`, `boolean`, `symbol`, `undefined`, `object`, `function`

`Array.isArray()`, `Object.getPrototypeOf()`

Discriminated Union: `switch (shape.kind) { case "circle": ... }`

weitere Typprüfungen: `instanceof` (`HttpError`), `if (var === null)`

`type Status = "idle" | "loading" | "success" | "error"`;

`const [status, setStatus] = useState<Status>("idle")`;

`type ButtonProps = { label: string; onClick: () => void; }`;

`// Using the type Alias to type the props`

```
export function Button({ label, onClick }: ButtonProps) {
  return <button onClick={onClick}>{label}</button>;
}
```

? (Optionales Property/Parameter) ist implizit `...` | `undefined`

```
function greet(name?: string) { return `Hello, ${name || 'stranger'}`; }
```

Typescript prüft die **Struktur**, nicht die Benennung

```
type Ball = { diameter: number; }; type Sphere = { diameter: number; };
let ball: Ball = { diameter: 10 }; let sphere: Sphere = { diameter: 20 };
sphere = ball; ball = sphere; // korrekt
```

Any deaktiviert Typ-Checking

Unknown kann alles sein und muss überprüft werden (mit `typeof` oder `parse`, etc). Type Checking während Runtime nennt man **Type Narrowing**.

```
function printValue(value: string | number) {
  if (typeof value === "string") {
    console.log(value, "toUpperCase()"); // value : string
  } else {
    console.log(value, toFixed(2)); // value : number
  }
}
```

Never Bedeutet der Wert soll nicht existieren. (z.B. Rückgabewert bei Exceptions) **Undefined** Eine Variable wurde deklariert, ihr aber nie ein Wert zugewiesen. **Null** Expliziter Wert, der sagt, das ist kein Objekt oder Wert

Generics: Platzhalter für Typen

```
function Identity<T>(x: T): T { return x; }
// Generics werden auch von useState und Array verwendet:
const [name, setName] = useState<string | null>('alice');
const names: string[] = []; // Kurzschreibweise
const names: Array<string> = []; // Langschreibweise
```

REACT ROUTING MIT REACTROUTER

Mit `React Router`: `<Route path="/" element={<Home />} />` Kann: Mehrere Seiten darstellen es sieht dann aus wie eine *traditionelle Website*, inklusive Änderung der URL, aber ohne *Reloads* (*Client Side Routing*), `URL` parameter und `Query Strings` verwenden, `Browser History` und `Navigation` verwenden.

```
<BrowserRouter> <!-- BrowserRouter im Idealfall in main.tsx platzieren -->
  <Routes>
    // Index Routes: renders into <Outlet> at parent's URL
    <Route index element={<Home />} /> // Index equals path="/"
    <Route path="about" element={<About />} />
    // Nested Routes: use <Outlet> in AuthLayout to render child
    <Route index element={<AuthLayout />} >
      // and Layout Routes: does not add segments to URL
      <Route path="login" element={<Login />} />
      <Route path="register" element={<Register />} />
    </Routes>
    // Route Prefixes: A <Route path> without element prop = path prefix to
    child routes, without parent layout.
    <Route path="concerts" >
      <Route index element={<ConcertsHome />} />
      // Dynamic Route / Route Params: becomes dynamic segment
      <Route path="city" element={<City />} /> // {city} = useParams();
      <Route path="trending" element={<Trending />} />
    </Route>
    // catch all/splat/start: ends with *
    <Route path="files/*" element={<File />} />
    // let params = useParams(); // params["*"] remaining URL
    // let filePath = params["*"];
  </Routes>
</BrowserRouter>
```

USEEFFECT & API CALLS

idrekt sofort nach dem ersten Rendern: `useEffect` hat noch nicht geladen, beachte auch `timeout` im code

CONTEXT API

Einige interessante Anwendungsfälle für useContext: Automatische Heading-Einstufung, Informationen zum angemeldeten User, Globaler State für Clientside-only Anwendungen, Localization, App Settings, Light/Dark Theme (auch in reinem CSS lösbar)

useContext zur Vermeidung von Prop-drilling?

Nicht immer: in typischen React Anwendungen gerne ein Duzend Props pro Komponente zusammenkommen, und das es üblich ist, über diese einige Stufen zu übergeben. ein expliziter Datenfluss ist an sich nicht schlechtes.

Lösungansätze für Prop-Drilling

1. State näher an Nutzung verschieben
2. Komponentenstruktur verbessern (Refactoring) (siehe auch nächste Slide)
3. Intentional weiterhin Props verwenden
4. Context API verwenden
5. WebStorage (Session/Local) + Custom Hooks verwenden (mit/ohne Context)
6. Zusätzliche State Management Libraries (zustand, Redux, ...)

```
// vorher
const Layout = ({ posts }) => {
  return (
    <div className="Layout">
      <Header />
      <Posts posts={posts} />
    </div>
  );
}

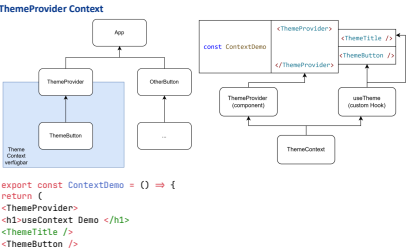
// --- Verwendung
<Layout posts={posts} />
```

Context API

`// Erstellen des Context`

```
const UserContext = createContext<InitialValue>
// Bereitstellen des Kontexts im Komponenten-Tree
// Der Value kann zum Beispiel von einem Prop oder State kommen
<UserContext.Provider value={user}>
  ...
</UserContext.Provider>
// Verwenden des Kontexts
const context = useContext(UserContext);
```

ThemeProvider Context



`</ThemeProvider>`

`</ThemeProvider>`

Warum die Unterteilung in zwei Files? Fast refresh only works when a file only exports components. Use a new file to share constants or functions between components. `eslint` (react-refresh/only-export-components) Der Grund ist, dass das Hot Module Reloading von Vite kaputtgeht, wenn man mehr in Komponentenfiles (.tsx) Funktionen exportiert.

Setup:

```
;
const ThemeTitle = () => {
  // Es müssen nicht alle Elemente
  // des Context verwendet werden:
  const { theme } = useTheme();
  return <h2>Current theme in title: {theme}</h2>;
};

const ThemeButton = () => {
  const { theme, toggleTheme } = useTheme();
  return (
    <button onClick={toggleTheme}>
      Current theme: {theme} (click to toggle)
    </button>
  );
};
```

Implementation Context

```
// theme-context.ts
export const ThemeContext = createContext
<ThemeContext.Provider value={undefined}>
  undefined,
);
export const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error("useTheme must be used
      within a ThemeProvider");
  }
  return context;
};
// theme-provider.tsx
export const ThemeProvider = (
  { children }:
  { children: ReactNode }
) => {
  const [theme, setTheme] = useState<Theme>('light');
  const toggleTheme = () => {
    setTheme((prev) =>
      (prev === 'light' ? 'dark' : 'light')
    );
  };
  return (
    <ThemeContext.Provider
      value={{ theme, toggleTheme }}
    >
      {children}
    </ThemeContext.Provider>
  );
};
```

CSS

Trennung von Page- und Komponentenlayouts: `styles/base.css` (resets, typography), `variables.css` (tokens, colors), `layout.css` (top-level page layout)

Cascades vermeiden, stattdessen Composition nutzen

```
:root { --color-primary: #0070f3; } /* Definition in der variables.css */
/* Nutzung in einer anderen Datei: */
.button { background-color: var(--color-primary); } Hier wird der Wert #0070f3
eingesetzt */
```

```
Composition via classname
.button {
  <!-- Basic-Styles -->
  <!-- primary -->
  <!-- Styles für Primär-Button -->
  <!-- large -->
  <!-- Styles für große Buttons -->
}
.button {
  padding: 10px; border-radius: 4px;
  background: var(--color-primary);
  font-size: 1.2em;
  font-weight: bold;
  color: white;
  text-align: center;
  width: 100%;
  height: 40px;
  line-height: 40px;
  margin: 0;
}
```

CSS Nesting: Verschachtelung ist ok, muss aber «flach» gehalten werden (maximal 3 Ebenen tief).

Einheiten: Verwenden konsequent `rem` anstelle von `px`, für Barrierefreiheit (Skalierbarkeit)

Gruppierung: Deklarationen innerhalb einer Regel sollten in einer logischen Reihenfolge stehen (z.B. erst Positionierung, dann Box-Modell wie `Display/Padding`, dann `Farben/Text`).

Shorthands vermeiden: Nutze keine unleserlichen Shorthands wie `padding: 1.2rem 0.8rem 1rem; margin: 0; border: 1px solid black;`

Gap & Padding: Nutze stattdessen `flexbox` oder `grid` in Kombination mit `gap` für Abstände zwischen Elementen und `padding` für Abstände innerhalb von Elementen.

EXPRESSJS

EXPRESS ARCHITEKTUR UND GRUNDLAGEN

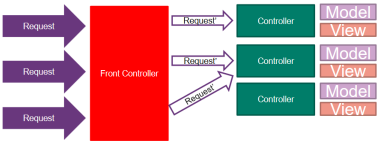
Controller und Middleware

MVC Pattern: Model (DB) View (hbs) und Controller (index.js, app.js) getrennt

Front-Controller Pattern:

Middleware Pattern: Verkettung von Middlewares pro Request, gemäss "Chain of Responsibility" Pattern.

jede Middleware: ist für genau eine Aufgabe verantwortlich, kann eine Aufgabe ausführen, muss den Request beenden oder weitergeben **Chain of Responsibility** is a behavioral design pattern that lets you pass requests along a chain of handlers. Upon receiving a request, each handler decides either to process the request or to pass it to the next handler in the chain.



Was ist CSR?: Client Side Rendering, rendert im Browser, bei Client, React

Vorteile und Nachteile CSR: schnellere Reaktion, weniger Arbeit auf Server, interaktiv. **Nachteile:**

SEO, benötigt JS, Content-Seiten die selten ändern rendern neu.

typische Webseiten CSR: Games, Social Media, Google Earth

Was ist SSR?: fertige statische Website von Server geliefert an Client, ExpressJS

typische Webseiten SSR: Blogs, statische Webseiten, News Portal

Vorteile und Nachteile SSR: Performance, SEO, einfach, nicht mehrfach die gleiche Seite rendern.

siehe Vorteile CSR

Vorteil beim Redirect von POST /random nach GET /random?from=... gegenüber der direkten Darstellung der Daten in der POST-Route? > Formulardarstellung nur 1 mal programmieren, einheitlich, DRY = URL kann weitergegeben werden von GET, Lesesachen kann erstellt werden > Mit FS (neuladen) werden bei POST die Daten nochmals abgesendet (mit Warnung Popoup vom Browser). Bei GET wird nur die Seite neu geladen.

COOKIES UND EXPRESSJS SESSION

– Um nach dem Login die Session auf der Website "abzusichern"



Server: set-cookie:name=value;Expires=Wed, 09 Jun 2029 10:18:14 GMT Client: cookie:name=value; name=value2

– Cookies für "ExpressJS Session": app.use(session({ secret: '1234567', resave: false, saveUninitialized: true}));

```
const app = express();
app.use(cookieParser("secret"));
```

```
app.get('/cookieDemo/{*pLati}', function (req, res) {
  console.log(JSON.stringify(req.cookies));
  console.log(JSON.stringify(req.signedCookies));
  res.cookie('url', req.url);
  res.cookie('signedUrl', req.url, {signed: true});
});
```

```
if (req.cookies.url) {
  res.end('das war dein letzter Besuch:
  Cookie: ${req.cookies.url}
  SignedCookie: ${req.signedCookies.signedUrl || "----"};
} else { res.end("erster Besuch?!") }
});
```

```
app.listen(3000, function () {
  console.log('Listening on http://localhost:3000');
});
```

JSON WEB TOKEN (JWT) JSON-BASED OPEN STANDARD (RFC 7519).

– um eine API abzusichern (REST)

HTTP-Header: Authorization: Bearer <token>

Inhalt: Header, Payload (Claims), Signatur

HTTPS: Token nur über eine sichere Verbindung versenden

speichern: Token kann auch im Cookie abgelegt werden **mit httpOnly, secure, SameSite, Lebensdauer und Achtung CSRF**



AJAX MIT FETCH REPETITION

```
fetch(url, {
  method: method,
  headers: { 'Content-Type': 'application/json'},
  body: JSON.stringify(data),
  credentials: 'include', // cookies mitsenden
}).then(x => {
  return x.json();
}) // statt json() -> arrayBuffer(), blob(), formData(), text()
});
```

// Hilfsklasse für Request von Fetch API

```
const myHeaders = new Headers();
myHeaders.append('Content-Type', 'text/plain');
const myInit = {
  method: 'GET',
  headers: myHeaders,
  cache: 'default'
};
```

```
const myRequest = new Request('/example', myInit);
fetch(myRequest) /*...*/
```

MIDDLEWARES

Routing

Middleware befindet sich auf dem Express Objekt

```
import express from 'express'; const router = express.Router();
```

Wichtige Methoden

```
// Wird unabhängig vom der HTTP-Methode aufgerufen
router.all(path, [callback, ...] callback)
```

Wird aufgerufen, falls die jeweilige HTTP-Methode verwendet wurde

METHOD = all, get

path = /*, /*+ (optional), /*:id (wird in req.params.id gespeichert)

router.METHOD(path, [callback, ...] callback)

router.get('/', function(req, res){ res.send('hello world'); });

Es können mehrere Callbacks als Chain übergeben werden

```
router.get('/admin', ensureAdmin, renderAdmin);
```

```
router.get('/profile/:id', ensureUser, renderProfile)
```

Static-Middleware

Statische Files ausliefern (Es sind mehrere static-routes möglich)

```
app.use(express.static('public'))
```

Custom-Middleware

3 Parameter (request, response, next)

– next zeigt auf die nächste Middleware im Stack, kann aufgerufen werden, um die nächste Middleware aufzurufen.

```
function myDummyLoggerMiddleware(options = {}) {
  options = {timestamp: true, ...options};
  return function myInnerDummyLogger(req, res, next) {
    const timestamp = options.timestamp ? new Date().toISOString() + " " :
    "";
    console.log(`${timestamp}${req.method} ${req.url}`)
    next();
  }
}
```

Error-Middleware

– muss 4 Parameter haben, die letzte (Error) Middleware muss die Anfrage beenden

– aufrufen bei: next(new Error("-")), Promise.reject(new Error("-...")), throw new Error("-...")

```
app.use(function(err, req, res, next) {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});
```

VIEW MIT TEMPLATE ENGINE - HANDLEBARS

Statt `res.type('text/html');` `res.write("<html>");` `res.write("<p>...</p>");` ...
Template Engines nutzen die Daten und Template zu HTML kombinieren (Hbs, Pug) **Begriffe:** Layouts (Definieren wiederverwendbare Grundstrukturen), Partials/Components (Wiederverwendbare Template-Bausteine), Helpers (Hilfsfunktion, Erweiterung der Template-Sprache)

```
Beispiel Template:
render("template", {pizzaName: "Hawaii", _id: 3, state: "OK"});
<>Order-Infos</>
<{{if pizzaName}}>
<>Ordered Pizza: {{pizzaName}}</>
<{{if eq state "OK"}}>
  <form action="/orders/{{_id}}" method='post'><input type='hidden'
name='_method' value='delete'><input type='submit' value='Delete order'>
  </form>
<{{/if}}>
<{{/if}}>

render("template", {description: "List of some Movies", items: songs});
<figure><ul>
  <{{each items}}>{{ title = this.title = bezieht sich auf items
  <li><h3>{{title}}</h3><p>{{this.artist}}</p></li>
  <{{/each}}>
  </ul><figcaption><p>{{description}}</p></figcaption>
</figure>
```

Beispiel Layout - Trennung View und Controller

```
export class IndexController {
  index(req, res) { res.render("index", {data: "Hello World", dark: true}); };
}

// Layout-hbs:
<doctype html><html lang="en"><head>
<meta charset="UTF-8"><title>Pizza</title><{{if dark}}<style>body {background:
black; color: white; }</style><{{/if}}>
</head>
<body>{{(body)}}</body>
</html>
```

Beispiel als Express View

```
// 1. import express-handlebars
import exphbs from 'express-handlebars';
const app = express();
// 2. configure
const hbs = exphbs.create({
  extname: '.hbs',
  defaultLayout: 'default',
  helpers: { ...helpers }
});
```

```
// 3. set engine and global values
app.engine('hbs', hbs.engine);
app.set('view engine', 'hbs');
// 4. path to views
app.set('views', path.resolve('views'));
```

MODEL = SERVICE

Möglichkeiten, um Daten zu speichern: In Memory: Array, JSON, NoSQL-Datenbanken = Dokumentorientierte Datenbanken (nedb), SQL-Datenbanken

nedb NoSQL

```
// Datenbank Laden
import Datastore from 'nedb';
const db = new Datastore({filename: './data/order.db', autoload: true});

// Einfügen - Feld _id wird gesetzt: eindeutige ID von der Datenbank
const order = new Order({pizzaName, orderRedBy}); const newOrder = await
db.insertAsync(order);
console.log(newOrder._id) // e8381097APAYTIS

// Suchen - find oder findOne
const order = db.findOneAsync({_id: id});
```

```
// Updaten - z.B. Einzelne Werte ändern, Array von einem <document> anpassen,
ganzes Objekt ersetzen
await db.updateAsync({_id: id}, {$set: {'state': OrderState.DELETED}});
```

